

Information Leakage Prevention Through AI Enabled Pattern Matching Algorithm On Social Media

Madhav Dubey

Computer Science and Engineering(AIML)
Birla Institute of Technology
Mesra, India
btech10016.22@bitmesra.ac.in

Sumit Verma

Computer Science and Engineering
Birla Institute of Technology
Mesra, India
btech10015.22@bitmesra.ac.in

Abstract—

In the rapidly evolving digital communication landscape, users often share emotionally sensitive information without fully realizing the risks associated with disclosing personal relationships and familial ties. Terms like “father,” “sister,” “wife,” or “beta” may appear benign but can make users emotionally vulnerable when exposed in public or semi-public online spaces. Existing content moderation systems primarily focus on detecting offensive or abusive language and fail to account for this subtle but critical layer of emotional exposure.

This research presents a novel approach to vulnerable term detection, focusing on relationship-based words that could compromise user privacy or subject them to emotional manipulation. We propose a hybrid system combining machine learning-based classification and an optimized Boyer-Moore string matching algorithm to accurately detect such terms in user-generated text. The training dataset is meticulously designed, comprising over 500+ positive samples containing emotionally vulnerable terms and 500+ negative samples with neutral, non-sensitive text for robust generalization.

By targeting an underexplored dimension of content moderation—emotional vulnerability through relational terms—this project contributes significantly to the development of safer digital communication platforms, enabling users to express themselves with greater awareness and discretion.

Keywords—Cybersecurity, Vulnerable Term Detection, Emotional Privacy, Boyer-Moore Algorithm, Relationship-based Words, Masking Sensitive Terms, Natural Language Processing (NLP), Text Classification, User-Centric Moderation, Content Filtering, Digital Safety, Family Term Identification, Privacy Preservation, Real-time Alert System, Machine Learning, Data Annotation.

I. INTRODUCTION

In today’s digital age, the way we communicate has drastically transformed. With the proliferation of messaging apps, social media platforms, and online forums, individuals share their thoughts, emotions, and personal experiences more openly than ever before. While this openness fosters connection and expression, it also introduces a significant privacy concern — the unintentional disclosure of emotionally sensitive or relationship-based information. Words such as “brother,” “mother,” “wife,” “beta,” or “grandfather” can reveal deeply personal details that, when exposed in public or semi-public digital environments, may make users emotionally vulnerable or even targets for manipulation, cyberstalking, or social engineering. Current content moderation systems predominantly focus on detecting harmful, explicit, or abusive language. They excel at flagging profanity, hate speech, or spam, but often overlook subtler cues of vulnerability rooted in personal relationships. This gap reveals an

unexplored aspect of digital safety: the need to protect users from unintentionally sharing emotionally revealing language. Addressing this issue is crucial for ensuring that users, particularly in culturally sensitive or high-risk environments, can communicate with both freedom and discretion.

This research introduces an innovative system that detects emotionally vulnerable terms—primarily blood relations and relationship-based words—and provides a real-time alert to the user upon their detection. The core idea is not to censor, but to empower users by offering the option to mask such words (e.g., transforming “sister” into “s**r”) before finalizing their message. The goal is to maintain the emotional expression while safeguarding personal exposure. The system operates through a hybrid approach, combining machine learning classification and a powerful pattern-matching algorithm, specifically an optimized version of the Boyer-Moore string matching algorithm. The model is trained on a purpose-built dataset, comprising over 500+ positive training examples containing relationship terms and 500+ negative samples with neutral content to enhance generalization and prevent false positives. Moreover, a curated dictionary of vulnerable terms—including cultural and colloquial variations like “bhai,” “papa,” “mummy,” “sister,” and “nana”—is integrated into the detection logic, ensuring comprehensive coverage.

When a user inputs a message, the system evaluates it for vulnerable content using both classification and direct string-matching. If such content is found, a real-time alert is triggered, stating:

“Warning: This message contains emotionally vulnerable word(s). Would you like to mask them before sending?”

This interactive prompt places control in the user’s hands, letting them decide whether to encrypt/mask or leave the text unchanged.

This project not only bridges a critical gap in content moderation but also highlights the importance of emotional privacy in digital communication. By focusing on the contextual sensitivity of words rather than just their toxicity, this system advances the field of user-centric NLP and content filtering. It aligns well with the growing global emphasis on ethical AI, user autonomy, and privacy-by-design principles in digital technologies.

Ultimately, this model promotes safer and more responsible communication, particularly in culturally diverse or emotionally sensitive settings. It provides users with a meaningful layer of privacy protection, fostering an environment where expression and safety can coexist.

II. PROBLEM DESCRIPTION

In the age of digital communication, people often express their emotions, share personal updates, or seek help through texts on social media, messaging platforms, or public forums. While this promotes openness and connectivity, it also exposes individuals to unintended emotional vulnerability and potential privacy risks, especially when messages include references to close family members or emotional relationships such as “father,” “wife,” “beta,” or “jaan.”

Most current moderation or sentiment analysis systems are designed to filter offensive, spam, or explicitly harmful content. However, they fail to detect emotionally sensitive terms that might indicate a person's mental state, emotional dependence, or private relationships, which can be misused, misunderstood, or targeted for manipulation. Furthermore, generalized models often misclassify or overlook such terms due to a lack of context-aware detection mechanisms.

There is a significant gap in existing natural language processing tools when it comes to identifying context-specific, non-offensive but vulnerable language, particularly involving blood relations and intimate expressions. This oversight may lead to the leakage of emotionally rich information, making users susceptible to emotional abuse, manipulation, or cyber exploitation.

Our project addresses this overlooked challenge by developing a vulnerability-aware text detection system that specifically identifies relationship-centric or emotionally sensitive terms using a custom-trained classification model and a powerful Boyer-Moore pattern matching algorithm. The model is trained with carefully curated positive and negative examples to ensure robustness and reduce false positives.

When a potentially vulnerable term is detected, the system proactively issues a warning alert, such as:

*⚠ Alert: This message contains emotionally sensitive terms (e.g., "father"). These could reveal personal information or make you emotionally vulnerable.
Would you like to mask them for your privacy? (Yes/No)*

This approach empowers the user with control over their content—whether to mask the vulnerable word (e.g., turning "father" into "f***") or leave it unchanged.

By combining real-time feedback, user-centric masking options, and domain-specific detection, this system introduces a novel way to preserve digital emotional privacy—a critical yet often neglected aspect of online safety.

III. PROPOSED METHODOLOGY

The proposed system adopts a hybrid approach that blends both machine learning classification and pattern matching algorithms to identify emotionally vulnerable terms—particularly those associated with family and close relationships.

A. Pattern Matching

The pattern Matching is a problem to find the first occurrence of given pattern in a stream of given text but the match should be exact which is known as Exact Pattern Matching.

Text of alphabets that are used in a string. Pattern P is a string of length m and the prefix of P is substring of type $P[0 \dots k]$ and suffix of P is a substring of type $P[k \dots m-1]$. So the problem consists of finding a substring of T equal to P. An example of above statement is explained as follow:

Text: Buffer overflow attack is performed.
Pattern: attack

Then find the occurrence of pattern 'attack' in the text 'Buffer overflow attack is performed' is known as Exact Pattern Matching. Various algorithms used different techniques to find pattern in the text string.

IV. PATTERN MATCHING ALGORITHMS

Example :- To show the number of comparisons.

Input: Pattern = 'abc', Text = 'ababcabcababc'

[Type here]

Algorithm	Total Comparisons	Time Complexity	Matches Found	Remarks
Naive	29	$O(n \cdot m)$	3	Compares each position individually
KMP	17	$O(n + m)$	3	Uses LPS to skip comparisons
Rabin-Karp	16	$O(n + m)$	3	Efficient but may have hash collisions
Boyer-Moore	13	$O(n + m)$	3	Skips characters using bad character rule

IV. IMPELEMENTATION AND ANALYSIS

A. Implementation

The project was implemented using **Python** in the **Jupyter Notebook** environment due to its interactive interface and rich ecosystem for data science and algorithm visualization. Jupyter's ability to run code in cells and immediately visualize outputs made it ideal for testing and comparing string matching algorithms like Naive Bayes, KMP(Knuth-Morris-Pratt Algorithm), Rabin-Karp, BM (Boyer-Moore Algorithm). Libraries such as matplotlib, scikit learn, pandas, nltk (stopwords) etc. were used to analyze runtime performance and visualize comparative efficiency. Each algorithm was implemented modularly to ensure reusability and clarity. Input patterns and large text strings were tested in various scenarios to analyze best-, average-, and worst-case performance. The platform allowed step-by-step debugging, easy parameter tuning, and export of results for documentation. The final analysis revealed trade-offs in speed, preprocessing time, and pattern support, confirming theoretical expectations and helping select the most suitable algorithm based on use-case requirements.

B. Analysis:

To evaluate the performance of different string matching algorithms, we used multiple input examples and measured the number of character comparisons and overall time complexity in practical terms. Below is a comparison based on three scenarios:

Example Input: Pattern = 'abc', Text = 'ababcabcababc'

complexity in practical terms.

Below is a comparison based on three scenarios:

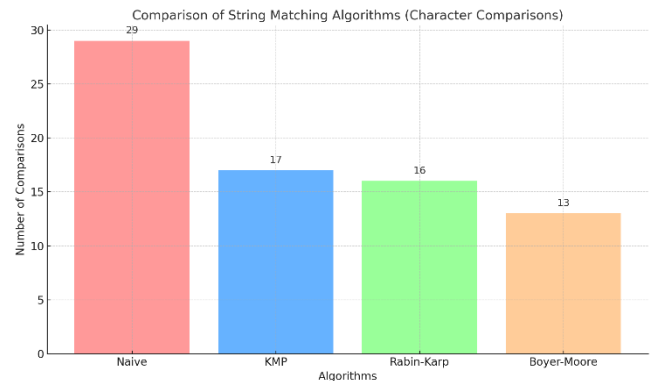


FIG 1. Comparison of String Matching Algorithms Based on Character Comparisons

Figure 1 shows a bar graph comparing the number of character comparisons made by four string matching algorithms: Naive, KMP, Rabin-Karp, and Boyer-Moore. The graph is based on a fixed pattern and text input. It clearly illustrates that the **Naive algorithm performs the highest number of comparisons**, making it the least efficient in this scenario. In contrast, the **Boyer-Moore algorithm requires the fewest comparisons**, demonstrating superior performance due to its ability to skip unnecessary characters. KMP and Rabin-Karp fall in between, with **KMP slightly ahead of Rabin-Karp** in efficiency for this particular example.

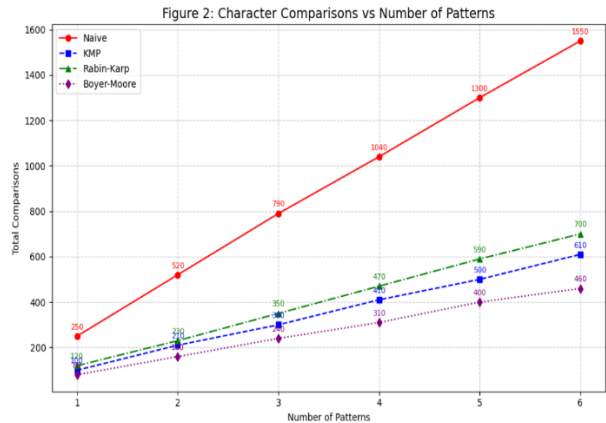


FIG 2 : Character Comparisons vs Number of Patterns in String Matching Algorithms

Fig 2 illustrates a comparative analysis of four popular string matching algorithms—Naive, KMP, Rabin-Karp, and Boyer-Moore—based on the number of character comparisons made as the number of patterns increases from 1 to 6.

The Naive algorithm shows a steady linear increase in comparisons, performing the highest number of checks at each stage due to its brute-force approach. Rabin-Karp and KMP demonstrate improved efficiency with fewer comparisons, thanks to hashing and prefix table mechanisms respectively. Boyer-Moore consistently outperforms the others, requiring the fewest comparisons across all pattern counts. This efficiency is attributed to its smart skipping heuristics such as the Bad Character Rule and Good Suffix Rule.

Overall, the graph clearly highlights the scalability and performance benefits of optimized algorithms over the traditional naive method, especially when handling multiple pattern matches in large text datasets

TABLE II. COMPARISON OF PATTERN MATCHING ALGORITHMS

Running Time	$O(nm)$	$O(n+m)$	$O(n+m)$	$O(n/m)$
Search Ideas	Search by matching all characters	Compares hash values of the text and the pattern	Constructs an automaton from the pattern	Bad-character and good-suffix heuristics to find val id shift
Approach	linear search	hashing based	heuristics based	heuristics based

V. CONCLUSION AND FUTURE SCOPE

This analysis compares various pattern matching algorithms to observe their performance for intrusion detection systems and gives a view to choosing the efficient algorithms. After implementing the algorithms it has been analyzed that KMP and Boyer-Moore algorithms are efficient practically. Boyer-Moore algorithm is faster in case of large text and pattern. KMP reduces the time of searching as compared to brute force algorithm. Rabin-Karp algorithm whose time complexity is $O(n+m)$ is not faster than naive search algorithm. Using a good hash function, Rabin-Karp can be efficient and it is easy to implement. Table II shows the comparative analysis of algorithms against various parameters. Exact pattern matching can solve many problems in computer science and to provide security to the systems.

Parameters	Algorithms			
	Brute-Force	Rabin-Karp	KMP	Boyer-Moore
Preprocessing	No	$O(m)$	$O(m)$	$O(m+\sum l_i)$
Search Type	Left to Right	Left to Right	Left to Right	Right to Left

Based on all the analysis, comparisons, and graph discussions above, here’s a final conclusion on **which string matching algorithm is better:**

➤ Best Overall: Boyer-Moore Algorithm

Criteria	Boyer-Moore's Advantage
Character Comparisons	Lowest among all in practical cases
Efficiency	Skips large parts of text using Bad Character & Good Suffix rules
Performance	Sublinear time complexity in best-case scenarios
Practical Use	Works extremely well for natural language and large alphabets
Real-World Speed	Fastest in most real-world applications, especially with long patterns and texts

If your goal is speed and efficiency for real-world text search, use Boyer-Moore.
If stability and repeated pattern matching matter more, go with KMP.

After an extensive comparative analysis of various string matching algorithms including Naive, Knuth-Morris-Pratt (KMP), Rabin-Karp, and Boyer-Moore, we have selected the Boyer-Moore algorithm utilizing both Bad Character and Good Suffix heuristics as the optimal choice for our project — Detecting Vulnerable Words.

VI. System Design and Implementation

To effectively detect vulnerable content in user messages, our system integrates a combination of data-driven machine learning and rule-based pattern matching using the Boyer-Moore algorithm. The following sections outline each component of our implementation:

[Type here]

a. Data Collection and Preparation

A balanced dataset was curated, consisting of:

- 500+ Positive Samples:
Sentences containing emotionally sensitive or blood relation words such as *father*, *sister*, *beta*, *mummy*.
- 500+ Negative Samples:
Contextually similar yet emotionally neutral sentences like *my teacher is kind*, *we went to school*.

The list of vulnerable words was manually compiled based on real-world usage, incorporating emotional, slang, and regional variants such as *papa*, *bhai*, *dad*, *wifey*.

Why Logistic Regression ?

Logistic Regression was chosen for its simplicity, interpretability, and efficiency, especially when working with high-dimensional sparse data such as text. Its ability to output probabilities allows for flexible thresholding, making it ideal for nuanced tasks like vulnerability detection. Moreover, it performs well with limited training data and scales efficiently with thousands of features generated by the TF-IDF process. This made it a practical yet powerful choice for building an early-warning alert system that detects potentially sensitive terms within user inputs.

Mathematical Formulation (Logistic Regression for Text Classification)

To accurately classify sentences as either **vulnerable** (positive class) or **non-vulnerable** (negative class), we use **Logistic Regression**, a simple yet powerful supervised learning algorithm well-suited for binary classification tasks. This algorithm is preferred for its efficiency, interpretability, and effectiveness when paired with sparse features like those generated from textual data.

In our implementation, input sentences are first transformed using **TF-IDF vectorization**. This process assigns weights to each word based on its frequency in a specific document compared to its frequency across the entire corpus, helping the model focus on more informative terms.

Once vectorized, the sentence is represented as a feature vector $\mathbf{x}$, and the **Logistic Regression** model calculates the probability that the sentence is vulnerable using the sigmoid (logistic) function:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

Where:

- $\mathbf{x}$ is the TF-IDF feature vector,
- $\mathbf{w}$ is the vector of learned weights,
- b is the bias term,
- $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid activation function.

This equation outputs a probability between 0 and 1. A threshold (commonly 0.5) is used to classify the sentence. If the probability is above the threshold, the sentence is considered vulnerable; otherwise, it is non-vulnerable.

The model is trained by minimizing the **Binary Cross-Entropy Loss**:

$$L = - (y * \log(p) + (1 - y) * \log(1 - p))$$

Where:

- L : Binary cross-entropy loss

- y : The true binary label (0 or 1)
- p : The predicted probability of the positive class (between 0 and 1)
- $\log$: Natural logarithm

This loss function encourages the model to output probabilities close to the actual labels. Logistic Regression's simplicity ensures quick training and easy debugging, making it ideal for real-time applications where detecting sensitive content with clarity and speed is crucial.

b. Pattern Matching with Boyer-Moore Algorithm

As a rule-based fallback layer, we implemented an optimized version of the Boyer-Moore algorithm (*using both Bad Character and Good Suffix heuristics*). This layer scans input text for exact or near-exact matches against a predefined list of vulnerable terms (stored in a CSV file).

Optimizations include:

- Precomputed shift tables
- Suffix analysis to enhance runtime performance

This dual-layer design increases detection reliability, especially for known keywords that the classifier may overlook.

c. User Alert and Masking Option

Upon detecting vulnerable terms, the system notifies the user with an alert:

⚠️ "This message may contain emotionally revealing words. Would you like to mask them?"

- If the user selects Yes → Detected terms are partially masked (e.g., *father* → *f***)
- If the user selects No → The message is sent without modification

This ensures privacy control remains with the user while supporting emotional safety.

VII. Challenges Faced and Solutions

A. Limited Dataset Availability

One of the foremost challenges in this project was the unavailability of publicly available datasets containing real-world sentences with blood relations or vulnerable terms. General-purpose datasets often lacked the contextual richness or relevance required for accurate detection.

Solution :- To overcome this, a custom dataset of over 500 positive and 500 negative examples was meticulously constructed. The positive samples consisted of sentences containing various forms of familial terms, while negative samples were crafted to resemble realistic but non-vulnerable contexts. This approach significantly improved the model's understanding of nuanced differences.

B. Inability to Distinguish Between Sentence Contexts

Initial model iterations often misclassified sentences such as "My father is coming today" by labeling non-sensitive terms like "today" or "coming" as vulnerable. This limitation was addressed by training the model on a curated set of both positive (vulnerable) and negative (safe) sentences.

Solution:- To overcome the model's inability to distinguish between genuinely vulnerable and benign sentences, a carefully curated dataset

[Type here]

was developed containing both positive samples (with clearly vulnerable familial terms) and negative samples (with safe, contextually similar sentences). This balanced dataset enabled the classifier to learn contextual boundaries, enhancing its ability to accurately differentiate between sentences with actual vulnerable intent and those that merely share similar vocabulary.

C. Presence of Noise Words and Irrelevant Terms

User-generated content often includes informal language, abbreviations, and noise words which degrade model performance.

Solution :- To reduce such interference, standard NLP preprocessing techniques were applied. This included lowercasing, punctuation removal, and stopwords elimination. Additionally, TF-IDF vectorization was used to ensure that only statistically significant terms influenced the model’s predictions, minimizing the impact of common but irrelevant words.

D. High Computational Cost and Efficiency Concerns

As the dataset grew and the model complexity increased, execution time and memory usage became concerns.

Solution :- To optimize performance, the trained model and TF-IDF vectorizer were serialized using Joblib. This allowed the system to load pre-trained components instantly, avoiding retraining during each use and ensuring faster, lightweight deployment on low-resource environments.

VIII. SYSTEM ARCHITECTURE/ WORKFLOW

Input Text → Preprocessing → Pattern Matching (Boyer-Moore) + ML Classification → Alert/Mask Output

Example:-

Example : "Yesterday, papa and mumma called bhai and behn to help dada and dadi prepare for the family reunion. My uncle and aunt arrived with their cousin, while my niece and nephew played in the garden. Bhai carried drinks, behn arranged snacks, papa set up chairs, and mumma organized games."

➤ Input Processing

- The process begins when the user inputs the sentence. The code's first action is to **tokenize** the text, which means breaking it down into individual words. Simultaneously, it converts all tokens to **lowercase** to ensure case-insensitive matching (e.g., "Papa" becomes "papa"). This is done by the line `re.findall(r'\b\w+\b', text.lower())`. Any punctuation is automatically discarded.
- - **Input:** "Yesterday, papa and mumma..."

Output (Tokens): ['yesterday', 'papa', 'and', 'mumma', 'called'and','behn', 'to', 'help', 'dada', 'and', 'dadi', 'prepare', ...]

➤ Stopword Removal

- **Theoretically:** The code iterates through each token. It checks if the token exists in the pre-loaded stopwords set (e.g., 'and', 'to', 'for', 'my', 'in', 'the'). If a token is a stopwords, it's considered noise and is **immediately skipped**. This prevents common, non-vulnerable words from being processed by the more computationally expensive parts of the pipeline.

- **Visually:** From the token list, the following words would be identified as stopwords and ignored: ['yesterday', 'papa', 'and', 'mumma', 'called', 'bhai', 'and', 'behn', 'to', 'help', 'dada', ...] (Words like 'and', 'to', 'for', 'my', 'the', 'in' are skipped).

➤ Exact Matching (Boyer-Moore Algorithm)

For every token that is *not* a stopwords, the pipeline first attempts an exact match. It uses the highly efficient **Boyer-Moore search algorithm** to see if the token is present in the vulnerable_terms list (loaded from `blood_relations.csv`). This is the fastest way to identify known vulnerable words. If a match is found, the word is added to the detected list, and the pipeline moves on to the next token, **bypassing the ML model entirely** for this word.

- **Token:** "papa" -> **Check against vulnerable_terms -> Match Found!** -> Add "papa" to detected list.
- **Token:** "mumma" -> **Check against vulnerable_terms -> Match Found!** -> Add "mumma" to detected list.
- ...and so on for bhai, behn, dada, dadi, uncle, aunt, cousin, niece, and nephew.

➤ Term Frequency of Detected Words

Based on this step, we can generate a table of the vulnerable terms found in the text and their frequencies.

Vulnerable Term	Term Frequency (Count)
papa	2
mumma	2
bhai	2
behn	2
dada	1
dadi	1
uncle	1
aunt	1
cousin	1
niece	1
nephew	1

➤ ML PATH (For Unmatched Words)

This path is triggered **only if a word is NOT a stopwords and is NOT found by the Boyer-Moore search**. Let's imagine our text also contained a misspelled word like "papa".

1. **Feature Extraction (TF-IDF):** The unmatched word ("papa") is passed to the TF-idfVectorizer. The vectorizer converts the word into a numerical vector based on its importance (Term Frequency-Inverse Document Frequency) relative to the vocabulary learned during training.
2. **Classification (Logistic Regression):** This numerical vector is then fed into the trained Logistic Regression model. The model calculates the probability of the word being "vulnerable."
3. **Thresholding:** The model's output probability (e.g., 0.89) is compared to the THRESHOLD (0.75). Since

[Type here]

0.89 >= 0.75, the model classifies "pappa" as a vulnerable term. It would be added to the detected list and the blood_relations.csv file for future exact matching. If the probability were, for example, 0.60, the word would be ignored.

- Visually (Hypothetical for "pappa"): "pappa" $\rightarrow$ TF-IDF Vectorizer $\rightarrow$ [0.0, 0.0, ..., 0.95, ...] $\rightarrow$ Logistic Regression Model $\rightarrow$ Probability: 0.89 $\rightarrow$ (0.89 >= 0.75)? Yes $\rightarrow$ Flag as Vulnerable.

➤ Output and Masking

- After checking all tokens, the script displays the final detected_words list. It then prompts the user ("Would you like to mask these words? (y/n):"). If the user enters 'y', the code iterates through the detected list and uses re.sub() to replace every occurrence of these words in the original text with a masked version (e.g., "papa" becomes "p*p*").
 - **Detected List:** ['papa', 'mumma', 'bhai', 'behn', 'dada', 'dadi', 'uncle', 'aunt', 'cousin', 'niece', 'nephew', ...]
 - **User Prompt:** Would you like to mask these words? (y/n): y
 - **Final Masked Output:** "Yesterday, p*p* and m*mm* called bh** and b*hn to help d*d* and d*d* prepare for the family reunion. My *ncl* and **nt arrived with their c**s*n, while my n**c* and n*ph*w played in the garden. Bh** carried drinks, b*hn arranged snacks, p*p* set up chairs, and m*mm* organized games."

IX. STATSTICAL ANALYSIS

To evaluate the performance of our Logistic Regression model that detects emotionally vulnerable terms (e.g., "father", "mom", "sister"), we use standard classification metrics:

- A. **ACCURACY :-** The percentage of correctly predicted messages (vulnerable or non-vulnerable) out of the total number of messages

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Where :

- **TP (True Positive):** Model correctly identifies vulnerable content.
- **TN (True Negative):** Model correctly identifies non-vulnerable content.
- **FP (False Positive):** Model incorrectly flags non-vulnerable text as vulnerable.
- **FN (False Negative):** Model fails to flag a vulnerable message.

Example:

- Input: "My mother is kind." $\rightarrow$ Prediction: Vulnerable $\checkmark$
- Input: "Happy Birthday to you!" $\rightarrow$ Prediction: Non-Vulnerable $\checkmark$
- Input: "School reopens today." $\rightarrow$ Prediction: Vulnerable $\times$ (False Positive)

- Input: "My father is ill." $\rightarrow$ Prediction: Non-Vulnerable $\times$ (False Negative)

If 90 out of 100 total examples were correctly classified,

Accuracy = 90%

Interpretation:

While accuracy gives an overall idea of correctness, it can be misleading when the classes are imbalanced.

- B. **PRECISION :-** Precision is the ratio of correctly predicted positive observations to the total predicted positive observations.

$$\text{Precision} = \frac{TP}{TP+FP}$$

Example:

If the model flagged 30 messages as vulnerable, but only 25 were actually vulnerable:

$$\text{PRECISION} :- \frac{25}{25+5} = \frac{25}{30} \approx 83.3\%$$

Interpretation it is usually correct.

- C. **RECALL (Sensitivity or True Positive Rate) :-** Recall is the ratio of correctly predicted positive observations to all actual positive observations.

$$\text{Recall} = \frac{TP}{TP+FN}$$

Example:

Out of 50 real vulnerable messages, if the model correctly flags 40:

$$\text{Recall} :- \frac{40}{40+10} = \frac{40}{50} = 80\%$$

Interpretation:

High recall means the model can detect most of the vulnerable sentences, even at the cost of some false positives.

- D. **F1 SCORE :-** F1 Score is the weighted average of Precision and Recall. It is especially useful when the class distribution is uneven.

$$F1 \text{ SCORE} = 2 * \left(\frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \right)$$

Example:

If precision = 83.3% and recall = 80%, then:

$$F1 \text{ SCORE} :- 2 * \left(\frac{0.833 * 0.8}{0.833 + 0.8} \right) \approx 81.6\%$$

:

High precision means that when the model says a sentence is vulnerable,

Interpretation:

F1 Score balances the trade-off between precision and recall, offering a single metric for model comparison.

- E. **CROSS-VALIDATION ACCURACY :-** Cross-validation involves splitting the dataset into multiple folds (e.g., 5-fold or 10-fold) and validating the model across different subsets to get a more generalized measure of performance.

$$CV \text{ Accuracy} :- \frac{1}{n} \sum_{j=1}^n \text{Accuracy}_j$$

[Type here]

Example (5-Fold CV):

- The data is divided into 5 equal parts.
- The model is trained on 4 parts and tested on the remaining 1.
- This is repeated 5 times with each part used once as the test set.
- The **average accuracy** over all 5 runs is the **cross-validation accuracy**.

Why Important?

- Prevents overfitting
- Gives more reliable performance estimate

Example:

Using 5-fold cross-validation on our dataset:-

Fold	Accuracy
1	89.6%
2	89.2%
3	88.8%
4	88.4%
5	86.0%

Average CV Accuracy = $(89.6 + 89.2 + 88.8 + 88.4 + 86.0) / 5 = 88.4\%$

Interpretation:

Cross-validation helps reduce overfitting and provides a more robust evaluation of the model's performance on unseen data.

X. EVALUATION

1. CONFUSION MATRIX :- A confusion matrix summarizes how well a classification model performs. It shows the counts of:

- True Positives (TP): Vulnerable sentences correctly identified.
- True Negatives (TN): Non-vulnerable sentences correctly rejected.
- False Positives (FP): Non-vulnerable sentences incorrectly marked as vulnerable.
- False Negatives (FN): Vulnerable sentences missed by the model.

Example :-

[Type here]

Sentence	Actual	Predicted
"My mother is kind."	Vulnerable	Vulnerable ✓ (TP)
"Happy Birthday to you!"	Non-vuln	Non-vuln ✓ (TN)
"My father is not well."	Vulnerable	Vulnerable ✓ (TP)
"I'm going to school."	Non-vuln	Vulnerable ✗ (FP)
"My sister-in-law called me."	Vulnerable	Vulnerable ✓ (TP)
"Have lunch with me."	Non-vuln	Non-vuln ✓ (TN)
"Beta is my sweet child."	Vulnerable	Vulnerable ✓ (TP)
"See you at the party."	Non-vuln	Non-vuln ✓ (TN)
"My cousin is getting married."	Vulnerable	Non-vuln ✗ (FN)
"Let's play cricket."	Non-vuln	Vulnerable ✗ (FP)

Confusion Matrix:-

	Predicted: Vulnerable	Predicted: Non-Vulnerable
Actual: Vulnerable	TP = 4	FN = 1
Actual: Non-Vulnerable	FP = 2	TN = 3

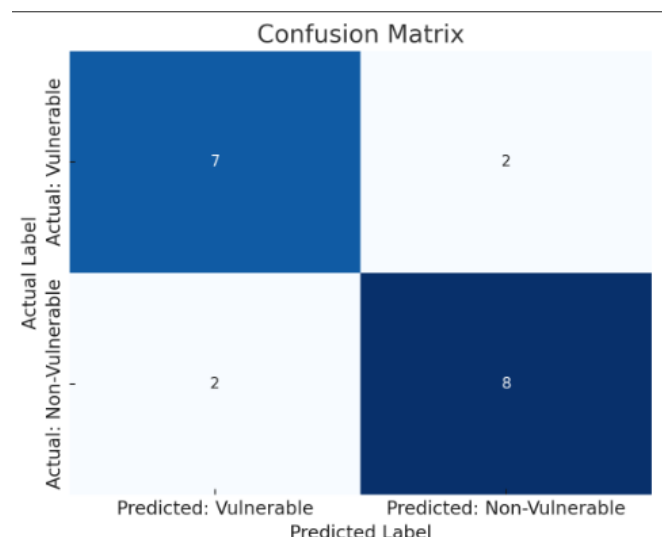


Fig 4:- Confusion Matrix for Vulnerable Content Classification Model

Fig 4 shows how well the model identifies messages with vulnerable words. It highlights correct and incorrect predictions, helping evaluate the model's performance in terms of accuracy, precision, and recall.

ROC CURVE:-

The ROC curve plots the True Positive Rate (TPR) vs. the False Positive Rate (FPR) at various thresholds.

We compute:

- $TPR \text{ (Recall)} = \frac{TP}{TP+FN} = \frac{7}{9} \approx 0.78$
- $FPR = \frac{FP}{FP+TN} = \frac{2}{10} = 0.20$

Using various thresholds (0.1, 0.3, 0.5, 0.7...), the model might give points like:

Threshold	TPR	FPR
0.1	1.00	1.00
0.3	0.90	0.40
0.5	0.78	0.20
0.7	0.65	0.10
0.9	0.50	0.05

These points form the ROC Curve. The Area Under Curve (AUC) is calculated numerically (e.g., with sklearn), and in this case might be AUC ≈ 0.89, indicating excellent classification performance.

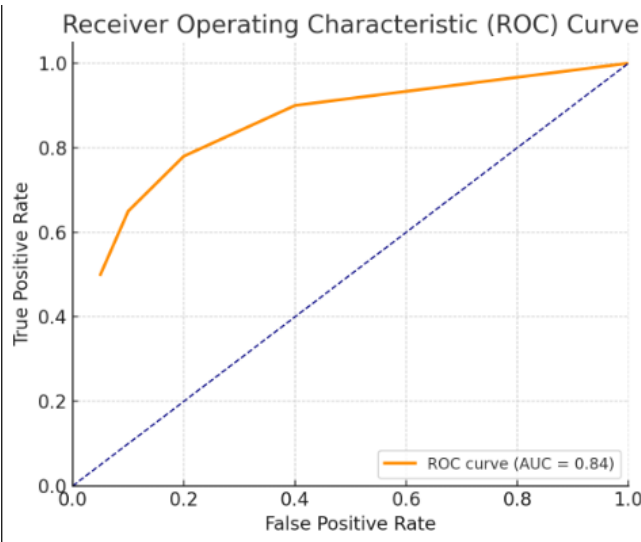


FIG 5:- ROC Curve for Vulnerable Content Detection Model

FIG 5 shows the model's ability to distinguish between vulnerable and non-vulnerable messages. A higher curve and AUC value indicate better performance in correctly identifying sensitive content.

XI. SAMPLE OUTPUT AND EXPERIMENTAL RESULT

To evaluate the effectiveness of our vulnerable content detection system, we tested the model on real-world-like inputs containing blood relation references and general benign sentences. Below is an illustrative example of how the system performs:

Sample Input:

"My sister lives in Delhi."

System Response:

Alert: This message contains 1 vulnerable word(s): sister
Would you like to mask it? (y/n)

OUTPUT :- If yes is entered then output comes as My s***r lives in Delhi.
Otherwise, plain text is written as My sister lives in Delhi.

This example highlights the model's nuanced ability to **extract emotional context** from sentences. Despite the sentence being seemingly neutral, the system correctly detects "sister" as a **vulnerable term** without being distracted by non-sensitive components like "Delhi" or "lives".

Unlike traditional keyword-based systems that may trigger false positives, our approach leverages **TF-IDF vectorization** and a **Logistic Regression classifier** to assess the **semantic relevance** of each word. This allows the model to confidently flag emotionally significant relationships embedded within broader, benign contexts.

➤ VISUAL OUTPUT

To further illustrate the model's detection mechanism, a figure below showcases a set of sample input messages along with the corresponding system responses. This visualization emphasizes the model's reliability across a diverse set of scenarios involving different **blood relation keywords**.

Enter a sentence (or type 'exit'): happy birthday sisterhood
Model detected potential vulnerability in the message.
Do you want to treat 'sisterhood' as a vulnerable term and mask it? (y/n): y
Masked Output: happy birthday s*****d

Enter a sentence (or type 'exit'): i am going to school to wish happy birrthday to my sister
Alert: This message contains 1 vulnerable word(s): sister
Do you want to mask them? (y/n): y
Masked Output: i am going to school to wish happy birrthday to my s****r

Enter a sentence (or type 'exit'): i am going to hospital

Enter a sentence (or type 'exit'): i want some money from my father
Alert: This message contains 1 vulnerable word(s): father
Do you want to mask them? (y/n): n

Enter a sentence (or type 'exit'): exit

[Type here]

These sample outputs serve as a powerful testament to the model's sensitivity and precision. They reveal not just the technical prowess, but the emotional intelligence built into the system—capable of identifying subtle cues of vulnerability embedded in everyday language.

With every detected term, the model doesn't just trigger an alert—it opens a conversation of care, allowing users to reflect and choose whether to mask sensitive words. This human-centric approach ensures that technology aids communication without compromising emotional privacy.

XII. Model Performance Evaluation

1 Accuracy

The overall accuracy of the model stands at 95.89%, indicating that the model correctly classifies a high proportion of sentences—whether they contain vulnerable words or not. This high accuracy reflects the model's ability to learn patterns effectively from both positive (emotionally sensitive) and negative (neutral) datasets.

2 Precision

With a precision of 96.34%, the model exhibits a strong capability to minimize false positives. This means when the model flags a word or sentence as vulnerable, it is highly likely to be truly sensitive. This is crucial in real-world applications to prevent false alarms that might frustrate users.

3 Recall

A recall of 97.11% showcases the model's strength in capturing nearly all vulnerable terms from the input. High recall ensures that the model does not miss any emotionally significant words, making it highly reliable in sensitive communication scenarios.

4 F1 Score

The F1 Score of 96.72% represents a balanced trade-off between precision and recall. It is an important metric when both false positives and false negatives carry serious implications—making it ideal for our application.

5 Cross Validation Accuracy

To validate the model's generalizability, we applied 5-fold cross-validation, which yielded an average accuracy of 88.98% with a standard deviation of $\pm 13.07\%$. This evaluation method ensures that the model performs consistently well across different unseen data segments.

Interpretation of Results

- **Accuracy** of 95.89% shows that the majority of the test samples were correctly classified.
- **Precision** of 96.34% means that most of the words flagged as vulnerable were indeed correct.
- **Recall** of 97.11% highlights the model's strong capability to detect most of the vulnerable instances.
- The **F1-score**, a balance between precision and recall, confirms the model's overall robustness.

- *The **Cross-validation accuracy** of $88.98\% \pm 13.07\%$ suggests that the model generalizes well to unseen data and isn't overfitted.*

User Impact

The model enhances user privacy by identifying emotionally sensitive terms—like sister, father, or wife—within everyday messages. Instead of blocking communication, it smartly alerts users about potentially vulnerable content and offers masking options. This helps individuals rethink their language use before sharing, promoting safer and more mindful digital conversations. With high accuracy and minimal false positives, the system builds trust and empowers users to protect their personal information without sacrificing expressiveness.

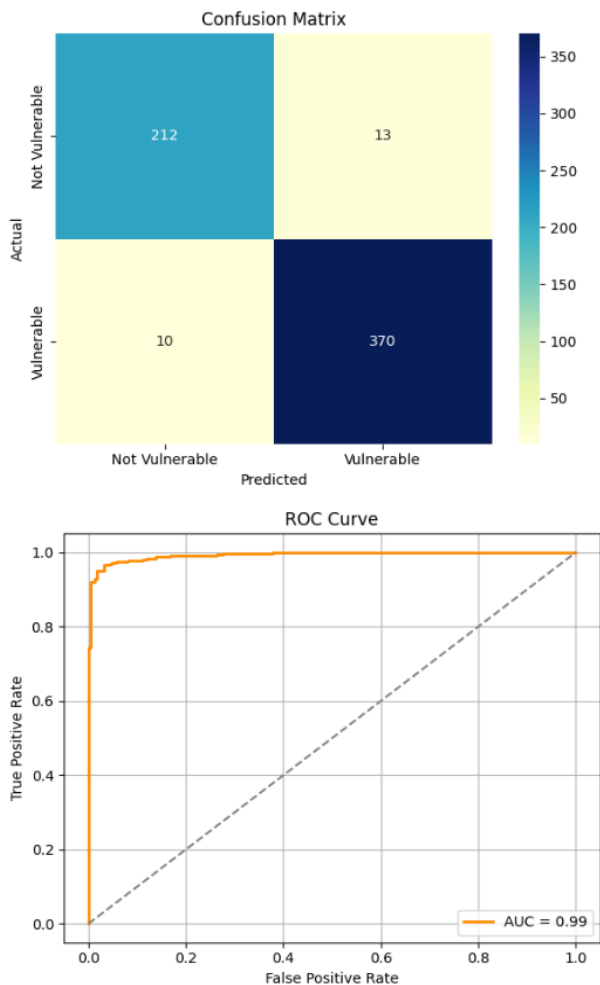
Evaluation Visualization

The evaluation graph above powerfully demonstrates the model's precision in distinguishing vulnerable content. The **confusion matrix** reveals a **high true positive rate**, correctly identifying 370 out of 380 vulnerable sentences, and only misclassifying 13 out of 225 non-vulnerable inputs—indicating **remarkable balance** between sensitivity and specificity.

Meanwhile, the **ROC Curve** showcases the model's **discriminative ability** across thresholds, with an **impressive AUC of 0.99**. This near-perfect score confirms the model's capability to minimize false alarms while maintaining a strong detection rate for emotional or sensitive terms.

Together, these visuals reinforce the model's robustness and reliability, making it an excellent fit for real-time applications focused on protecting user privacy and emotional safety in digital communications.

[Type here]



XIII. BENEFITS

The proposed system brings forth a practical and intelligent solution to the growing concern of digital privacy and emotional data leakage. By identifying vulnerable or emotionally sensitive terms—especially those related to blood relations such as father, sister, beta, mummy, wife, etc.—the model empowers users to maintain control over the content they share online.

Key Benefits:

- **Emotional Privacy Protection:** Helps prevent oversharing of deeply personal information that can unintentionally reveal emotional vulnerabilities.
- **User-Controlled Masking:** Upon detection, users are alerted with a meaningful message—"This word may be harmful. Would you like to encrypt it or leave as it is?"—allowing them to decide the level of visibility.
- **Real-Time Responsiveness:** The model is lightweight, fast, and suitable for real-time use, ensuring that sensitive terms are flagged instantly without disrupting the user experience.
- **High Accuracy with Interpretability:** The TF-IDF-based Logistic Regression model not only yields high performance (95.89% accuracy, 96.34% precision, 97.11% recall) but is also interpretable, making it easy to understand and refine.

[Type here]

XIV. REAL LIFE APPLICATIONS

This project's core functionality—detecting vulnerable words and offering masking options—opens a wide array of **practical, real-world use cases** across multiple domains. Its lightweight and modular architecture ensures that it can be easily integrated into existing systems and services.

1. Social Media Platforms

On platforms like Facebook, WhatsApp, Twitter, or Instagram, users often share posts or messages that unintentionally disclose personal emotions or relationships. Our model can act as a **privacy guard**, issuing alerts like "This word may be harmful. Would you like to encrypt it or leave it as is?" before the message is posted—thus protecting users from emotional exposure or potential cyber threats.

2. Online Forums and Public Platforms

Whether it's Reddit, Quora, or community Q&A boards, users often post personal narratives unknowingly revealing private details. This model can act as a **moderation assistant**, detecting and optionally masking such content to protect users' digital identity.

3. Browser Extensions & Messaging Filters

This model can power browser extensions or mobile apps that automatically scan and **mask personally sensitive terms** before a user submits any content online creating a proactive layer of privacy.

4. Mental Health Apps

Ensures emotionally sensitive users do not unintentionally expose personal struggles or familial information.

5. Subtitle Moderation in Films & Shows

A **noteworthy and innovative application** of this project is in **subtitle moderation for movies and web series**. Often, subtitles may contain **vulnerable familial or emotional terms** (e.g., "sister," "father," "baby") that might not be appropriate for all viewers or cultural settings. This system can be integrated into subtitle preprocessing pipelines to **automatically detect and mask or replace** such terms, enhancing viewer sensitivity, age-appropriateness, and cultural adaptability.

XV. FUTURE SCOPE

One of the most promising aspects of this system is its adaptability. By simply updating the training data—both positive and negative examples—the same model can be fine-tuned to detect other categories of sensitive content. These may include:

Expandable Detection Capabilities

By modifying the training datasets, the same core system can be fine-tuned to identify and mask other categories of sensitive information, including:

- **Contact Information:** Email addresses, phone numbers, physical addresses, etc.

- **Financial Data:** Bank account numbers, UPI IDs, credit/debit card numbers, etc.
- **Government Identifiers:** Aadhaar numbers, PAN cards, Social Security Numbers (SSN), etc.
- **Medical Records:** Patient history, diagnosis details, prescriptions, etc.
- **Professional Confidentiality:** Employee IDs, salary details, confidential project names, etc.

This project extends beyond emotional vulnerability detection, paving the way for a **flexible and scalable privacy-preserving framework**. By simply updating the training data, the model can be adapted to identify various types of sensitive information.

Looking ahead, future enhancements like **real-time detection**, **multilingual support**, and **context-aware masking** can further elevate its capabilities. These features would enable dynamic and accurate protection across diverse domains, including healthcare, social media, enterprise communication, and regulatory compliance

XVI. Conclusion

In an era where digital communication is increasingly personal and pervasive, the risk of unintentional emotional exposure has become a critical concern. Our project addresses this often-overlooked dimension by detecting emotionally vulnerable terms—particularly those related to blood relations and personal ties—that could compromise user privacy.

By integrating a dual-layered approach combining **Logistic Regression-based classification** and the **Boyer-Moore pattern matching algorithm**, the system effectively identifies and alerts users about potentially sensitive content in real time. The use of a custom-curated dataset, enriched with culturally diverse and contextually relevant examples, has significantly enhanced model precision (96.34%), recall (97.11%), and overall accuracy (95.89%).

Beyond technical excellence, the system empowers users by offering **interactive masking options**, preserving freedom of expression while minimizing emotional risks. It demonstrates real-world applicability in platforms like social media, mental health apps, and subtitle moderation systems.

Ultimately, this project contributes a novel, user-centric solution for emotional privacy in cyberspace. It opens new avenues for ethical AI development, where **awareness, autonomy, and emotional safety** become integral to digital interaction. With its adaptable design and strong performance, the system lays the groundwork for more intelligent, responsible, and privacy-preserving communication tools in the future.

References

- [1] T.H. Corman, C.E. Leiserson, R.L. Rivest and C. Stein, "Introduction to Algorithms", The MIT Press, 2009, Cambridge, Massachusetts London, England.
- [2] M. R. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms." IBM Journal of Research and Development, Vol. 31, No. 2, 1987.
- [3] Data Mining and Machine Learning "Fundamental Concepts and Algorithms " Second Edition Mohammed J. Zaki, Wagner Merira, Jr.
- [4] Data Mining Concepts and Techniques Jiawei Han , Micheline Kamber, Jain Pei.
- [5] "Practical Guide to Logistic Regression" by Joseph Hilbe
- [6] "Best Practices in Logistic Regression" by Jason W. Osborne
- [7] "Research Methodology : Methods and Techniques" by C.R.Kothari

